

BÀI 6. LÀM VIỆC VỚI ENTITY FRAMEWORK CORE

Bài trước, chúng ta đã xây dựng xong chức năng giỏ hàng, nhưng toàn bộ dữ liệu sản phẩm vẫn đang là "hàng giả" (hard-code). FakeProductRepository đã hoàn thành xuất sắc nhiệm vụ của nó, nhưng giờ là lúc để nó "nghỉ hưu". Bài này, chúng ta sẽ học cách sử dụng **Entity Framework Core (EF Core)** - một "nhà ngoại giao" tài ba - để ứng dụng C# của chúng ta có thể "nói chuyện" được với CSDL SQL Server một cách trôi chảy.

MỤC TIÊU BÀI HỌC

Sau khi hoàn thành tuần học này, bạn sẽ có khả năng:

- **Giải thích được khái niệm ORM** và vai trò của nó như một "người phiên dịch" giữa C# và SQL.
- **Trình bày rành mạch** vai trò của DbContext (phiên làm việc) và DbSet (đại diện cho bảng).
- **Tự tay cài đặt và cấu hình** EF Core để kết nối ứng dụng với CSDL SQL Server.
- **Trải nghiệm khoảnh khắc "aha!"** khi thay thế hoàn toàn nguồn dữ liệu của ứng dụng chỉ bằng cách thay đổi **một dòng code duy nhất**.

NỘI DUNG LÝ THUYẾT CHI TIẾT

6.1. Giới thiệu ORM và Entity Framework Core

6.1.1. Vấn đề "bất đồng ngôn ngữ"

Hãy tưởng tượng, ứng dụng của bạn (viết bằng C#) và CSDL (dùng SQL) đến từ hai "quốc gia" khác nhau và nói hai ngôn ngữ khác nhau:

- **Thế giới Lập trình Hướng đối tượng (C#):** Tư duy bằng Objects, Classes, Properties, Lists.

```
//C#
Product p = new Product();
p.Name = "Bóng đá";
```

- **Thế giới CSDL Quan hệ (SQL):** Tư duy bằng Tables, Columns, Rows, Relations.

```
//SQL
INSERT INTO Products (Name) VALUES ('Bóng đá');
```

Nếu không có "người phiên dịch", bạn sẽ phải tự tay viết các chuỗi lệnh SQL bằng C#, thực thi chúng, rồi lại tự tay đọc từng dòng, từng cột kết quả để chuyển đổi ngược lại thành đối tượng C#. Việc này cực kỳ **nhàm chán, tốn thời gian, và dễ sinh lỗi**.

6.1.2. ORM (Object-Relational Mapping) - "Người phiên dịch" thiên tài

ORM ra đời để giải quyết chính xác vấn đề "bất đồng ngôn ngữ" này. Nó là một lớp phần mềm trung gian, một "người phiên dịch" tự động làm tất cả công việc nặng nhọc cho bạn.

- **Mapping (Ánh xạ):** Nó "nhìn" vào class Product của bạn và tự động hiểu rằng nó tương ứng với một bảng Products trong CSDL.
- **Translation (Phiên dịch):** Khi viết một câu lệnh LINQ đơn giản trong C#:

```
//C#
var expensiveProducts = context.Products.Where(p => p.Price > 100);
```

- ORM sẽ tự động dịch nó thành câu lệnh SQL tối ưu:

```
//SQL
SELECT * FROM Products WHERE Price > 100
```

6.1.3. Lợi ích của ORM & Giới thiệu EF Core

- **Tăng năng suất:** Bạn tập trung vào logic nghiệp vụ thay vì viết SQL.
- **An toàn hơn:** Giảm thiểu nguy cơ bị tấn công **SQL Injection** vì các truy vấn được tham số hóa tự động.
- **Độc lập CSDL:** Về lý thuyết, bạn có thể chuyển từ SQL Server sang PostgreSQL mà chỉ cần thay đổi cấu hình, không cần sửa code truy vấn.

Entity Framework Core (EF Core) là ORM chính thức, hiện đại, và mạnh mẽ nhất do Microsoft phát triển cho hệ sinh thái .NET. Nó là công cụ tiêu chuẩn ngành mà chúng ta sẽ sử dụng.

6.2. DbContext và DbSet - "Cánh cổng" và "Người đại diện"

Để làm việc với EF Core, chúng ta cần hiểu hai khái niệm cốt lõi.

6.2.1. DbContext - "Cánh cổng" làm việc với CSDL

Hãy tưởng tượng DbContext như là **cánh cổng chính** hoặc một **phiên làm việc (session)** của bạn với toàn bộ CSDL.

- **Quản lý kết nối:** Nó chịu trách nhiệm mở và đóng kết nối đến CSDL.
- **Cấu hình ánh xạ:** Nó cho EF Core biết class nào tương ứng với bảng nào.
- **Theo dõi thay đổi (Change Tracking):** Đây là tính năng cực kỳ quan trọng. Khi bạn lấy một đối tượng Product từ CSDL, thay đổi giá của nó, DbContext sẽ "ghi nhận" sự thay đổi này. Khi bạn gọi SaveChanges(), nó sẽ biết chính xác cần tạo ra câu lệnh UPDATE nào để cập nhật xuống CSDL.

6.2.2. DbSet<T> - "Người đại diện" cho một bảng

Nếu DbContext là cánh cổng vào CSDL, thì DbSet<T> là **người đại diện** cho từng bảng cụ thể bên trong đó.

- Một thuộc tính public DbSet<Product> Products { get; set; } bên trong DbContext báo cho EF Core rằng: "Trong CSDL này có một bảng Products, và mỗi dòng trong bảng đó có thể được ánh xạ thành một đối tượng Product".
- Bạn sẽ thực hiện tất cả các thao tác **CRUD (Create, Read, Update, Delete)** trên bảng thông qua DbSet này bằng các câu lệnh LINQ.

6.2.3. Áp dụng vào SportsStore

Tạo một class ApplicationDbContext kế thừa từ DbContext của EF Core.

```
//C# - Kế thừa từ DbContext để có tất cả các năng lực của EF Core
public class ApplicationDbContext : DbContext
{
    // Constructor này cho phép DI Container "tiêm" các cấu hình (như chuỗi
    kết nối) vào
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
        : base(options) { }
    // Dòng này ánh xạ class Product với một bảng trong CSDL.
    // EF Core sẽ tự động đặt tên bảng là "Products" (dạng số nhiều).
    public DbSet<Product> Products { get; set; }
}
```

6.3. Hoàn thiện Repository Pattern với EF Core

6.3.1. Nhắc lại sức mạnh của Repository Pattern

Ở Bài 2, chúng ta đã thiết kế HomeController chỉ phụ thuộc vào "bản mô tả công việc" IProductRepository, chứ không phụ thuộc vào "nhân viên" cụ thể FakeProductRepository.

```
//C#
public class HomeController : Controller
{
    private IProductRepository _repository; // Chỉ biết đến hợp đồng!
    public HomeController(IProductRepository repo) { ... }
    // ...
}
```

Đây là một quyết định kiến trúc cực kỳ sáng suốt. Nó tạo ra một hệ thống **cắm-rút (pluggable)**.

6.3.2. "Cắm" Repository mới vào

Bây giờ, chúng ta sẽ tạo một "nhân viên" mới, chuyên nghiệp hơn, biết làm việc với CSDL thật, tên là EFProductRepository. Nhân viên này cũng sẽ ký vào cùng một "bản hợp đồng" IProductRepository.

```
//C#
public class EFProductRepository : IProductRepository // Ký vào cùng hợp đồng
{
    private ApplicationDbContext _context; // Nhân viên này có một "chìa khóa" vào CSDL

    public EFProductRepository(ApplicationDbContext ctx)
    {
        _context = ctx;
    }

    // Thực thi hợp đồng: cung cấp danh sách sản phẩm.
    // Thay vì trả về list giả, giờ đây nó trả về DbSet thật từ CSDL.
    public IQueryable<Product> Products => _context.Products;
}
```

6.3.3. Thay đổi nguồn dữ liệu

Sau khi đã có EFProductRepository, việc "nâng cấp" toàn bộ ứng dụng từ dữ liệu giả lên CSDL thật chỉ đơn giản là vào Program.cs và thay đổi người nhân viên được giao việc.

- Code cũ:

```
//C#
builder.Services.AddScoped<IProductRepository, FakeProductRepository>();
```

- Code mới:

```
//C#
builder.Services.AddScoped<IProductRepository, EFProductRepository>();
```

Toàn bộ ứng dụng của bạn vừa "nâng cấp" từ dữ liệu giả lên CSDL thật mà **không cần sửa một dòng code nào trong Controller**. Đây chính là thành quả ngọt ngào của việc thiết kế một kiến trúc linh hoạt.

6.4. Hướng dẫn thực hành

6.4.1. Bước 1: Cài đặt các gói NuGet cần thiết

Cách 1: Cài đặt NuGet Packages trong VS Studio:

- **Thao tác:** Chuột phải vào project SportsStore.Infrastructure -> **Manage NuGet Packages....**
- **Cài đặt:**

1. Microsoft.EntityFrameworkCore.SqlServer

- **Giải thích:** Đây là "**trình điều khiển**" (**database provider**). Nó chứa code cụ thể để EF Core có thể "nói chuyện" được với CSDL SQL Server. Nếu bạn muốn dùng CSDL khác (như PostgreSQL, MySQL), bạn sẽ cài một gói tương ứng khác.

2. Microsoft.EntityFrameworkCore.Tools

- **Giải thích:** Gói này cung cấp các công cụ dòng lệnh (như Add-Migration, Update-Database) mà chúng ta sẽ sử dụng ở Bài 7 để quản lý cấu trúc CSDL. Cài đặt nó ngay bây giờ để chuẩn bị.

Cách 2: Cài đặt NuGet Packages trong VS Code/CLI

Vì VS Code là một trình soạn thảo mã nguồn nhẹ (code editor) chứ không phải một IDE "đầy đủ" (full-featured IDE) như Visual Studio, nên bạn sẽ không thực hiện thao tác "Chuột phải -> Manage NuGet Packages..." như trong Visual Studio.

Thay vào đó, bạn sẽ sử dụng Terminal (Thiết bị đầu cuối) tích hợp sẵn và sử dụng công cụ .NET CLI (Command-Line Interface) để thêm các gói NuGet vào project của mình.

1. Mở Terminal tích hợp

1. Trong VS Code, mở project SportsStore.Infrastructure.
2. Mở Terminal bằng cách vào menu Terminal New Terminal hoặc sử dụng phím tắt (`$\text{Ctrl} + \text{\texttt{'}}$` trên Windows/Linux, `$\text{Cmd} + \text{\texttt{'}}$` trên macOS).

2. Di chuyển đến thư mục Project

Bạn cần đảm bảo rằng Terminal đang trở đến thư mục chứa file .csproj của project SportsStore.Infrastructure.

```
//Bash
# Ví dụ, nếu bạn đang ở thư mục gốc của Solution:
cd SportsStore.Infrastructure
```

3. Cài đặt các gói NuGet

Sử dụng lệnh dotnet add package để cài đặt từng gói:

1. Cài đặt Microsoft.EntityFrameworkCore.SqlServer (Database Provider)

- o Mục đích: Để EF Core biết cách làm việc với SQL Server.

```
//Bash
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

2. Cài đặt Microsoft.EntityFrameworkCore.Tools (Công cụ Dòng lệnh)

- o Mục đích: Cung cấp các lệnh quản lý CSDL (như Add-Migration, Update-Database) mà bạn sẽ dùng sau này.

```
//Bash
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Lưu ý quan trọng: Đôi khi, khi sử dụng .NET CLI (Terminal trong VS Code), lệnh này yêu cầu tham số --tools hoặc phải cài đặt gói công cụ toàn cục. Tuy nhiên, cách phổ biến nhất để dùng các lệnh EF Core trong VS Code là cài đặt gói Tools vào project SportsStore.Infrastructure như trên, sau đó chạy các lệnh Migration (nhập ...) từ project SportsStore.

Sau khi chạy thành công hai lệnh trên, hai gói NuGet sẽ được thêm vào file .csproj của bạn, hoàn tất thao tác cần thiết.

6.4.2. Bước 2: Soạn thảo chuỗi kết nối trong appsettings.json

- **Thao tác:** Mở file appsettings.json trong project SportsStore.WebUI.
- **Code:**

```
//JSON
{
  "Logging": { ... },
  "AllowedHosts": "*",
  // Thêm section ConnectionStrings
  "ConnectionStrings": {
    // "SportsStoreConnection" là tên bạn đặt cho chuỗi kết nối này.
    "SportsStoreConnection":
    "Server=(localdb)\\MSSQLLocalDB;Database=SportsStore;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

- Giải phẫu chuỗi kết nối:
 - Server=(localdb)\\MSSQLLocalDB: Chỉ định địa chỉ của server CSDL. (localdb)\\MSSQLLocalDB là tên mặc định của phiên bản **SQL Server Express LocalDB**, một phiên bản gọn nhẹ cho lập trình viên được cài sẵn cùng Visual Studio.
 - Database=SportsStore: Tên của CSDL mà chúng ta muốn kết nối tới (EF Core sẽ tự tạo nếu nó chưa tồn tại).
 - Trusted_Connection=True: Sử dụng xác thực tích hợp của Windows để đăng nhập vào CSDL, không cần username/password.
 - MultipleActiveResultSets=true: Một tùy chọn kỹ thuật cho phép thực thi nhiều lệnh trên cùng một kết nối, thường được yêu cầu bởi EF Core.

6.4.3. Bước 3: Viết Code và Cấu hình DI

1. **Tạo ApplicationDbContext.cs** trong SportsStore.Infrastructure (như code đã phân tích ở trên).
2. **Tạo EFProductRepository.cs** trong SportsStore.Infrastructure (như code đã phân tích ở trên).
3. **Cập nhật Program.cs** trong SportsStore.WebUI.

```
//C#

// Thêm các using cần thiết ở đầu file

using Microsoft.EntityFrameworkCore;
using SportsStore.Infrastructure;
using SportsStore.Domain; // Cần thiết cho IProductRepository

var builder = WebApplication.CreateBuilder(args);
// ...

// --- BẮT ĐẦU CẤU HÌNH EF CORE ---

// Đăng ký dịch vụ DbContext với DI Container.
builder.Services.AddDbContext<ApplicationDbContext>(options => {
    // options.UseSqlServer(...) báo cho EF Core biết chúng ta sẽ dùng CSDL
    // SQL Server.
    // builder.Configuration.GetConnectionString("SportsStoreConnection")
    // sẽ tự động đọc chuỗi kết nối có tên "SportsStoreConnection" từ file
    // appsettings.json.

    options.UseSqlServer(builder.Configuration.GetConnectionString("SportsStore
    Connection"));
});

// --- THAY ĐỔI DÒNG NÀY ---
// Thay thế "nhân viên giả" bằng "nhân viên thật".
builder.Services.AddScoped<IProductRepository, EFProductRepository>();

// ...
```

CÂU HỎI LÝ THUYẾT CÙNG CỐ

1. Hãy giải thích vai trò của một ORM bằng ví dụ "người phiên dịch".
2. DbContext và DbSet khác nhau như thế nào? Cái nào đại diện cho toàn bộ CSDL, cái nào đại diện cho một bảng?
3. Tại sao HomeController không bị ảnh hưởng khi chúng ta thay FakeProductRepository bằng EFProductRepository? Nguyên lý thiết kế nào đã giúp chúng ta đạt được điều này?
4. Trong chuỗi kết nối, Server=(localdb)\MSSQLLocalDB có ý nghĩa gì?

TỔNG KẾT BÀI HỌC

Bài này, ứng dụng của chúng ta đã thực sự "trưởng thành":

- Chúng ta đã học cách **bắc một cây cầu vững chắc** giữa thế giới hướng đối tượng của C# và thế giới quan hệ của CSDL bằng EF Core.
- Chúng ta đã thấy được **sức mạnh thực sự của kiến trúc linh hoạt** mà mình đã xây dựng, cho phép thay đổi các thành phần cốt lõi một cách dễ dàng.
- Ứng dụng giờ đây đã sẵn sàng để làm việc với dữ liệu thật, mở đường cho các tính năng phức tạp hơn và việc triển khai lên môi trường thực tế.

BÀI TẬP VỀ NHÀ

1. **Thực hiện lại:** Tự tay thực hiện tất cả các bước: cài đặt NuGet, sửa appsettings.json, tạo DbContext, tạo EFProductRepository, và cập nhật Program.cs.
2. **Khám phá CSDL:** Sau khi chạy ứng dụng lần đầu (EF Core sẽ tự tạo CSDL), hãy mở **SQL Server Object Explorer** trong Visual Studio (View -> SQL Server Object Explorer). Tìm đến server (localdb)\MSSQLLocalDB, mở rộng Databases -> SportsStore -> Tables. Chuột phải vào bảng dbo.Products và chọn **View Data**. Bạn có thấy dữ liệu mẫu (nếu có seed) không?
3. **(Nâng cao)** Trong EFProductRepository, hãy thử viết một phương thức mới GetProductById(int id) sử dụng LINQ để tìm một sản phẩm duy nhất dựa trên ID của nó. (Gợi ý: dùng phương thức .FirstOrDefault()).

TÀI LIỆU THAM KHẢO

- [1]. **Introduction to Entity Framework Core:** [Microsoft Docs - EF Core](#)
- [2]. **Installing EF Core:** [Microsoft Docs - Installing EF Core](#)
- [3]. **DbContext Lifetime, Configuration, and Initialization:** [Microsoft Docs - DbContext Configuration](#)
- [4]. **Connection Strings:** [Microsoft Docs - Connection Strings](#)